

NAME: Ayilara Busari Dare

DATE: 2/12/2024

Student ID: IDEAS/24/28133

Lab 4: Simulating Network Routing and VLAN Configuration in Linux

Exercise 1: Setting Up Static Routing in Linux 1.

Task: Set up static routes in Linux to simulate routing behavior. Ensure the Linux machine can route traffic to the OWASP Broken Web Application VM.

- **Commands:** 1. `sudo ip route add via dev` Example: `sudo ip route add 192.168.4.0/24 via 192.168.1.1 eth0`

- 2. `ip route show` – Verify your routing table.

- **Verification Command:** 1. `ping` – Ensure that the Linux machine can reach the OWASP Broken Web Application VM.

2. Question:

- How does static routing work on a Linux system?

I used `sudo ip route add 192.168.92.0/24 via 10.0.2.2 dev eth0`

Static routing on a Linux system involves manually configuring routes to direct network traffic to specific destinations without relying on dynamic routing protocols. This is achieved by adding static route entries to the system's routing table, which instructs the operating system how to forward packets to particular networks or nodes. Static routes are especially useful when you need to control the path that data takes, ensuring it bypasses certain gateways or flows through specified routes. In simple term A static route is a pre-defined pathway that network traffic follows to reach a specific host or network. Unlike dynamic routing, static routes are manually set and do not change automatically if the network topology changes

- What challenges could arise when setting up routes manually?

1. **Configuration Errors:** Typos in IP addresses or subnet masks can lead to routing issues and unreachable destinations.
2. **Gateway Unreachability:** Incorrect or unreachable gateways can prevent packets from being delivered effectively.
3. **Routing Loops and Conflicts:** Improperly configured routes may create loops or overlap, causing congestion or incorrect routing behavior.

4. **Static Nature:** Static routes do not adapt to network changes, requiring manual updates when conditions change.
5. **Complexity in Large Networks:** Managing static routes in expansive networks can become cumbersome and challenging without proper documentation.
6. **Troubleshooting Difficulties:** Identifying routing issues can be complex, especially without integrated diagnostic tools.
7. **Security Risks:** Incorrect routes may inadvertently bypass security measures, exposing the network to vulnerabilities.

Exercise 2: VLAN Configuration Using Network Namespaces 3.

Task: Create virtual LANs (VLANs) using Linux network namespaces to segment traffic. Assign the OWASP Broken Web Application VM to a separate namespace and test connectivity between namespaces

• **Commands:** 1. `sudo ip netns add vlan1` – Create a network namespace for VLAN 1. 2. `sudo ip netns add vlan2` – Create a network namespace for VLAN 2. 3. `sudo ip link add veth1 type veth peer name veth2` – Create virtual network interfaces. 4. `sudo ip link set veth1 netns vlan1` – Assign veth1 to vlan1. 5. `sudo ip link set veth2 netns vlan2` – Assign veth2 to vlan2. 6. `sudo ip netns exec vlan1 ip link set lo up` – Enable loopback interface in the namespace. 7. `sudo ip netns exec vlan1 ip addr add 192.168.10.1/24 dev veth1` – Assign IP. 8. `sudo ip netns exec vlan1 ping <OWASP IP>` – Test communication.

4. Question:

- How do network namespaces simulate VLANs in Linux?

Network namespaces in Linux are powerful tools for simulating network environments, including VLANs. They allow you to create isolated network stacks, where each namespace has its own network interfaces, routing table, and ARP table. To simulate VLANs using network namespaces, virtual Ethernet (veth) pairs and VLAN tagging come into play.

```
File Actions Edit View Help
Engr.ABD@kali: ~ x Engr.ABD@kali: ~ x 168.92.135
Kali Linux - Kali Tools - Kali Docs - Kali Forums - Kali NetHunter - Exploit-DB - Google Gruyere
(Engr.ABD@kali)-[~]
$ sudo ip netns add vln2
(Engr.ABD@kali)-[~]
$ sudo ip link add veth1 type veth peer name veth2
(Engr.ABD@kali)-[~]
$ sudo ip link set veth1 netns vln1
(Engr.ABD@kali)-[~]
$ sudo ip link set veth2 netns vln2
(Engr.ABD@kali)-[~]
$ sudo ip netns exec vln1 ip link set lo up
(Engr.ABD@kali)-[~]
$ sudo ip netns exec vln1 ip addr add 192.168.92.135 dev veth1
(Engr.ABD@kali)-[~]
$ sudo ip netns exec vln1 ping 192.168.92.135
PING 192.168.92.135 (192.168.92.135) 56(84) bytes of data.
64 bytes from 192.168.92.135: icmp_seq=1 ttl=64 time=2.65 ms
64 bytes from 192.168.92.135: icmp_seq=2 ttl=64 time=0.038 ms
64 bytes from 192.168.92.135: icmp_seq=3 ttl=64 time=0.037 ms
64 bytes from 192.168.92.135: icmp_seq=4 ttl=64 time=0.029 ms
64 bytes from 192.168.92.135: icmp_seq=5 ttl=64 time=0.040 ms
64 bytes from 192.168.92.135: icmp_seq=6 ttl=64 time=0.038 ms
64 bytes from 192.168.92.135: icmp_seq=7 ttl=64 time=0.047 ms
64 bytes from 192.168.92.135: icmp_seq=8 ttl=64 time=0.030 ms
^C64 bytes from 192.168.92.135: icmp_seq=9 ttl=64 time=0.030 ms
64 bytes from 192.168.92.135: icmp_seq=10 ttl=64 time=0.026 ms
```

- What are the benefits of using namespaces in network isolation?

Network namespaces in Linux provide significant advantages for network isolation and management. They enable the complete separation of network resources, such as routing tables, interfaces, firewall rules, and IP stacks, ensuring that changes in one namespace do not impact others. This strong isolation makes namespaces ideal for multi-tenant environments or scenarios where resource separation is critical.

Namespaces are particularly useful for simulating complex network topologies, as they allow developers to create multi-node networks on a single host. This capability is valuable for testing configurations, protocols, and applications without requiring additional physical devices. Furthermore, namespaces enhance security by isolating processes at the network level, preventing applications in one namespace from accessing resources in another.

Compared to traditional virtualization methods, namespaces are lightweight and efficient because they are built into the Linux kernel. They do not introduce the overhead of virtual machines, making them ideal for scenarios where performance and resource efficiency are important. For developers, namespaces offer flexibility by enabling the creation of isolated environments for testing without affecting the host or other namespaces.

Namespaces are tightly integrated with containerization technologies like Docker and Kubernetes, forming the foundation for containerized network isolation. Each container can operate within its own namespace, ensuring isolated IP addressing and secure communication. Administrators can easily manage namespaces using Linux tools like `ip netns`, connecting them with virtual Ethernet pairs, bridges, or physical interfaces for advanced configurations.

In addition to their technical advantages, namespaces provide a cost-effective solution for building and simulating network environments. They allow for the creation of complex setups on a single machine, reducing the need for additional hardware. This is especially beneficial in education, training, and research, where low-cost, isolated environments are crucial.

By combining resource control, enhanced security, and ease of management, network namespaces empower developers and administrators to create robust, isolated network setups that are secure, scalable, and efficient.

Exercise 3: IP Address Assignment and Subnetting in Linux 5.

Task: Subnet your network and assign IP addresses to each namespace and the OWASP VM. Ensure the correct routing between subnets using static routes.

• Steps:

1. Subnet the given network (e.g., 192.168.0.0/24).
2. Assign IP addresses to network namespaces and devices.
3. Configure static routes to enable communication between the subnets.

• Commands: 1. `ip addr add <subnet ip> dev <interface>` – Assign IP addresses.

2. `ip route add subnet` – Add static routes.

6. Question:

- How does subnetting work in Linux environments?

Subnetting in Linux environments involves dividing a large IP network into smaller, more manageable subnetworks to optimize IP address utilization and improve network management. This is achieved through the use of subnet masks, which filter parts of the IP address to designate which portion refers to the network and which part refers to the host. By configuring IP addresses and their respective subnet masks, you can create logical subdivisions of networks, ensuring efficient allocation of addresses and managing network traffic effectively.

```

(Engr.ABD@kali)-[~]
$ ip route show
default via 10.0.2.2 dev eth0 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev eth0 proto kernel scope link src 10.0.2.15 metric 100
192.168.0.0/24 dev eth0 proto kernel scope link src 192.168.0.0

(Engr.ABD@kali)-[~]
$ sudo ip addr add 192.168.0.128/24 dev eth0
Error: ipv4: Address already assigned.

(Engr.ABD@kali)-[~]
$ sudo ip addr add 192.168.0.128/24 dev veth1
Cannot find device "veth1"

(Engr.ABD@kali)-[~]
$ sudo ip route add 192.168.0.0/24 via 10.0.2.2 dev eth0
RTNETLINK answers: File exists

```

- What challenges arise when configuring subnets manually?

Configuring subnets manually can lead to several challenges, including:

1. **IP Address Conflicts:** Manual configurations increase the risk of assigning the same IP address to multiple devices, which can cause network disruptions.
2. **Scalability Issues:** As network needs grow, managing IP address allocation dynamically can become complex and inefficient, making manual configurations cumbersome.
3. **Security Risks:** Manual subnets may have vulnerabilities, such as unauthorized usage of IP addresses and potential exposure to attacks if not properly controlled.

Exercise 4: Testing Connectivity Using Ping and Traceroute 7.

Task: Use ping and traceroute to test connectivity between the Linux machine, network namespaces, and the OWASP Broken Web Application VM. Analyze the routing path.

- Commands:
- ping – Check connectivity.
- traceroute – Trace the packet path.

8.Question:

- What can traceroute reveal about network issues?

Traceroute is a tool that helps you see the path your data takes across the network, from your device to the destination. The tool 'traces' in real time the 'route' data packets take on an IP network. This is useful for diagnosing network issues and pinpointing where data packets are getting lost.



```
Engr.ABD@kali: ~ x Engr.ABD@kali: ~ x 192.168.92.135
(Engr.ABD@kali)-[~]
$ traceroute 192.168.92.135
traceroute to 192.168.92.135 (192.168.92.135), 30 hops max, 60 byte packets
 1  10.0.2.2 (10.0.2.2)  3.127 ms 10.961 ms 11.943 ms
 2  * * *
 3  * * *
 4  * * *
 5  * * *
 6  * * *
 7  * * *
 8  * * *
 9  * * *
10  * * *
11  * * *
12  * * *
13  * * *
14  * * *
15  * * *
16  * * *
17  * * *
18  * * *
19  * * *
20  * * *
21  * * *
22  * * *
23  * * *
24  * * *
```

- How does packet routing affect network performance?

Packet routing significantly affects network performance by determining the path and efficiency with which data packets are transmitted across the network. Factors like the number of routers and the complexity of network topologies can lead to increased latency and potential congestion, as packets may require multiple conversions and reassembly, impacting overall speed. Additionally, issues in the packet path, such as errors or buffer congestion, can result in dropped packets and necessitate retransmissions, further degrading performance.

Exercise 5: Configuring iptables for Routing and Firewall Rules

Task: Use iptables to simulate a router by controlling the flow of traffic between different networks and the OWASP VM. Block certain traffic while allowing other traffic.

- **Steps:** 1. Enable IP forwarding on the Linux machine. 2. Set up iptables rules to allow or block specific traffic between namespaces and the OWASP VM. 3. Test the firewall by pinging the OWASP VM and other devices.
- **Commands:** 1. `echo 1 > /proc/sys/net/ipv4/ip_forward` – Enable IP forwarding.

2. `sudo iptables -A FORWARD -s 192.168.10.0/24 -d 192.168.20.0/24 -j ACCEPT` – Allow forwarding.

3. `sudo iptables -A FORWARD -s 192.168.10.0/24 -d -j DROP` – Block traffic to OWASP VM.

10.Question:

- How can iptables be used to simulate routing and firewall functionality?

iptables can simulate routing and firewall functionality by allowing administrators to create and manage rules for filtering IP packets in the Linux kernel. While iptables itself is not a router, it can be configured to act as one by directing packets based on specific rules, manipulating both incoming and outgoing traffic. This can be used to implement firewall functions by defining rules that allow, or block traffic based on criteria such as source/destination IP addresses and port numbers.

To simulate routing, iptables can manage packet forwarding and control route selection, effectively allowing the system to operate in a manner similar to a router under certain configurations. Additionally, by leveraging the hooks provided by the Net filter framework, iptables can encapsulate more complex routing behaviors while maintaining security through its filtering capabilities.

```
Engr.ABD@kali: ~
File Actions Edit View Help
(Engr.ABD@kali)-[~]
$ echo 1 > /proc/sys/net/ipv4/ip_forward
zsh: permission denied: /proc/sys/net/ipv4/ip_forward
(Engr.ABD@kali)-[~]
$ sudo echo 1 > /proc/sys/net/ipv4/ip_forward
zsh: permission denied: /proc/sys/net/ipv4/ip_forward
(Engr.ABD@kali)-[~]
$ sudo bash -c 'echo 1 > /proc/sys/net/ipv4/ip_forward'
[sudo] password for Engr.ABD:
(Engr.ABD@kali)-[~]
$ sudo iptables -A FORWARD -s 192.168.10.0/24 -d 192.168.92.135/24 -j ACCEPT
(Engr.ABD@kali)-[~]
$ sudo iptables -A FORWARD -s 192.168.92.135/24 -d 192.168.92.135/24 -j ACCEPT
(Engr.ABD@kali)-[~]
$ sudo iptables -A FORWARD -s 192.168.10.0/24 -d 192.168.10.0/24 -j ACCEPT
(Engr.ABD@kali)-[~]
$ sudo iptables -A FORWARD -s 192.168.10.0/24 -d 192.168.92.135 -j DROP
(Engr.ABD@kali)-[~]
$ iptables
iptables v1.8.10 (nf_tables): no command specified
Try `iptables -h' or 'iptables --help' for more information.
(Engr.ABD@kali)-[~]
$ iptables -l
iptables v1.8.10 (nf_tables): unknown option "-l"
Try `iptables -h' or 'iptables --help' for more information.
```

```
File Actions Edit View Help
iptables v1.8.10 (nf_tables): no command specified
Try `iptables -h' or 'iptables --help' for more information.
(Engr.ABD@kali)-[~]
$ iptables -l
iptables v1.8.10 (nf_tables): unknown option "-l"
Try `iptables -h' or 'iptables --help' for more information.
(Engr.ABD@kali)-[~]
$ iptables -l --line-number
iptables v1.8.10 (nf_tables): unknown option "-l"
Try `iptables -h' or 'iptables --help' for more information.
(Engr.ABD@kali)-[~]
$ iptables -L --line-numbers
iptables v1.8.10 (nf_tables): Could not fetch rule set generation id: Permission denied (you must be root)
(Engr.ABD@kali)-[~]
$ sudo iptables -L --line-numbers
Chain INPUT (policy ACCEPT)
num target prot opt source destination
Chain FORWARD (policy ACCEPT)
num target prot opt source destination
1 ACCEPT all -- 192.168.10.0/24 192.168.92.0/24
2 ACCEPT all -- 192.168.92.0/24 192.168.92.0/24
3 ACCEPT all -- 192.168.10.0/24 192.168.10.0/24
4 DROP all -- 192.168.10.0/24 192.168.92.135
Chain OUTPUT (policy ACCEPT)
num target prot opt source destination
$
```

- **What are common mistakes when configuring iptables rules?**

Common mistakes when configuring iptables rules include:

- **Rule ordering:** Rules may be ordered incorrectly
- **Rule complexity:** Rules may be too verbose or hard to understand
- **Rule count:** There may be too many rules
- **Allowing any service:** Rules may allow any service inbound or outbound

Exercise 6: Monitoring Traffic Using tcpdump

11. Task: Use tcpdump to monitor traffic between your Linux namespaces and the OWASP VM. Capture and analyze packets to understand traffic flow.

- **Commands:** • `sudo tcpdump -i` – Capture traffic on a specific interface.
- `sudo tcpdump -i veth1` – Monitor traffic on the virtual network interface.
- `sudo tcpdump -i eth0 src` – Monitor traffic to/from OWASP VM.

12.Question:

- **How can tcpdump be used to diagnose network issues?**

Tcpdump is a powerful packet capture tool that can be used to diagnose network issues by capturing and analyzing network traffic. It allows users to collect detailed information about data packets traversing a network, making it useful for troubleshooting connectivity problems, identifying network bottlenecks, and analyzing protocol behaviors. By examining the captured packets, administrators can gain insights into traffic patterns, potential packet loss, and unusual network activities.

```
File Actions Edit View Help
(Engr.ABD@kali)-[~]
└─$ sudo tcpdump -i eth0 src 192.168.250.135
[sudo] password for Engr.ABD:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
07:23:56.229486 IP 192.168.250.135.http > 10.0.2.15.33190: Flags [S.], seq 493568001, ack 4045335700, win 65535, options [mss 1460], length 0
07:23:56.245932 IP 192.168.250.135.http > 10.0.2.15.33190: Flags [.], ack 433, win 65535, length 0
07:23:56.276958 IP 192.168.250.135.http > 10.0.2.15.33190: Flags [P.], seq 1:399, ack 433, win 65535, length 398: HTTP: HTTP/1.1 304 Not Modified
07:24:06.303975 IP 192.168.250.135.http > 10.0.2.15.33190: Flags [.], ack 433, win 65535, length 0
07:24:11.282817 IP 192.168.250.135.http > 10.0.2.15.33190: Flags [.], ack 434, win 65535, length 0
07:24:11.284411 IP 192.168.250.135.http > 10.0.2.15.33190: Flags [F.], seq 399, ack 434, win 65535, length 0
07:24:30.631507 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [S.], seq 497728001, ack 2272596075, win 65535, options [mss 1460], length 0
07:24:30.643414 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [.], ack 461, win 65535, length 0
07:24:30.687516 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [P.], seq 1:709, ack 461, win 65535, length 708: HTTP: HTTP/1.1 301 Moved Permanently
07:24:30.720532 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [.], ack 922, win 65535, length 0
07:24:31.705513 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [P.], seq 709:3629, ack 922, win 65535, length 2920: HTTP: HTTP/1.1 200 OK
07:24:31.706871 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [P.], seq 3629:4033, ack 922, win 65535, length 404: HTTP
07:24:31.967449 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [.], ack 1331, win 65535, length 0
07:24:31.974072 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [P.], seq 4033:6953, ack 1331, win 65535, length 2920: HTTP: HTTP/1.1 200 OK
07:24:31.974825 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [P.], seq 6953:7378, ack 1331, win 65535, length 425: HTTP
07:24:31.988172 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [.], ack 1759, win 65535, length 0
07:24:31.994054 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [P.], seq 7378:8634, ack 1759, win 65535, length 1256: HTTP: HTTP/1.1 200 OK
07:24:32.136138 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [.], ack 2191, win 65535, length 0
07:24:32.141407 IP 192.168.250.135.http > 10.0.2.15.39338: Flags [S.], seq 497920001, ack 1487300577, win 65535, options [mss 1460], length 0
07:24:32.141408 IP 192.168.250.135.http > 10.0.2.15.39344: Flags [S.], seq 497984001, ack 4269666236, win 65535, options [mss 1460], length 0
07:24:32.145419 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [P.], seq 8634:9668, ack 2191, win 65535, length 1034: HTTP: HTTP/1.1 200 OK
07:24:32.170155 IP 192.168.250.135.http > 10.0.2.15.39350: Flags [S.], seq 498048001, ack 1196769177, win 65535, options [mss 1460], length 0
07:24:32.180331 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [.], ack 2618, win 65535, length 0
07:24:32.183940 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [P.], seq 9668:11128, ack 2618, win 65535, length 1460: HTTP: HTTP/1.1 200 OK
07:24:32.184554 IP 192.168.250.135.http > 10.0.2.15.39330: Flags [P.], seq 11128:11190, ack 2618, win 65535, length 62: HTTP
07:24:32.195288 IP 192.168.250.135.http > 10.0.2.15.39350: Flags [.], ack 432, win 65535, length 0
07:24:32.200847 IP 192.168.250.135.http > 10.0.2.15.39350: Flags [P.], seq 1:2921, ack 432, win 65535, length 2920: HTTP: HTTP/1.1 200 OK
07:24:32.218229 IP 192.168.250.135.http > 10.0.2.15.39350: Flags [P.], seq 2921:2924, ack 432, win 65535, length 3: HTTP
07:24:32.240993 IP 192.168.250.135.http > 10.0.2.15.39344: Flags [.], ack 432, win 65535, length 0
```

- What types of traffic do you observe during the simulation?

While monitor traffic from OWASP VM using tcpdump, traffic types observed seq, ack and http.